Saskatchewan Polytechnic

COMP258: Midterm Exam

2 Hours

October 27, 2026

Pages: 3 (including cover page)

This is an open book exam. You may use any resources you require **other** than chat rooms, messaging software, discussion forums, or other communication technology, including AI applications. If you are caught using communication technology and/or AI applications (listed above or otherwise), you will be asked to leave the exam and **receive a 0** on this assessment.

Academic dishonesty will NOT be tolerated. This exam is an individual assessment. **It must be your own work.** Sharing the contents of this exam or trying to use the answers provided by others who have shared this exam will not be tolerated. Sharing your own work or using the work of others will not be tolerated. If you do not submit your own work, you will receive a 0% on this assessment. If it is suspected that you did not submit your own work the department reserves the right to conduct an interview to assess your knowledge of your submission.

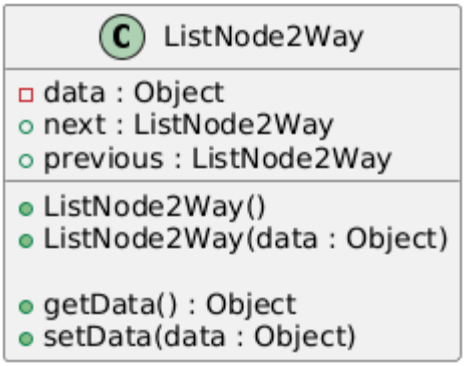
Part 1

Create a new project outside of your source-controlled folder. You will do all the work for this midterm in this new project, then zip it and submit it on Brightspace.

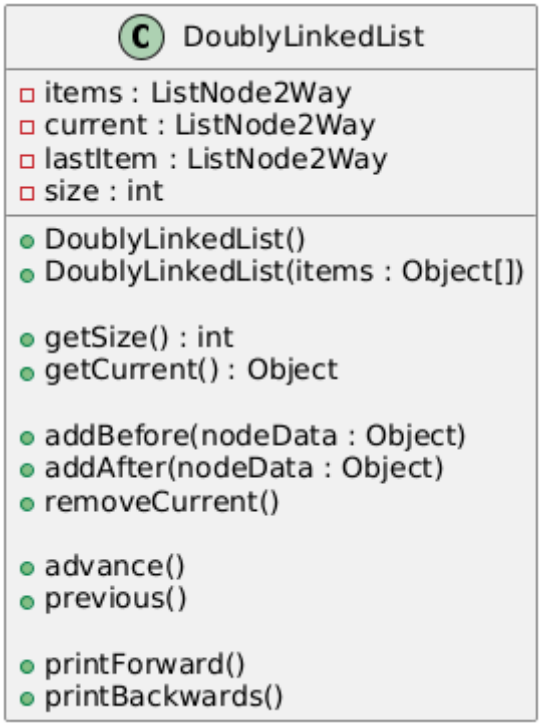
Note: you may find it useful to copy the code from your link list project into your midterm project to modify as much of the code you will write for the midterm will be quite similar.

So far in Comp 258, we have been using a Singly Linked List or a list of nodes that can only reference the node next in line. In this section it is your job to create a Doubly Linked List, where each node has reference to both the next and previous nodes as outlined in the UMLs below.

Create a ListNode2Way class that manages a collection of list nodes based on the following UML class diagram (3 marks):



Create a DoublyLinkedList class based on the following UML class diagram (12 marks):



The methods that add and remove items should update the previous pointers and the lastItem node when needed. Because you now have access to the previous node, the removeCurrent and addBefore methods should not require parsing through the list. Remember to test your edge cases.

Create a driver program for your DoublyLinkedList class. You may also use the provided ConsoleProgram Java file as a starting point for the app (8 marks).

Part 2

At the top of your driver program answer the following questions in a comment (7 marks):

Describe a specific situation where a pointer-based collection is a better choice than an index based one. (2 marks)

What about an index-based collection over a pointer based one? (2 marks)

If we were to add a parameter type <T extends <Comparable<T>> to this new list, what would that allow us to do with the list, or what assumptions could we make? (3 marks)